

PlayList Manager

Project 03 - Graduation Project

Presented by :
Nourhan Ahmed Kamal

Project Overview

Core Idea:

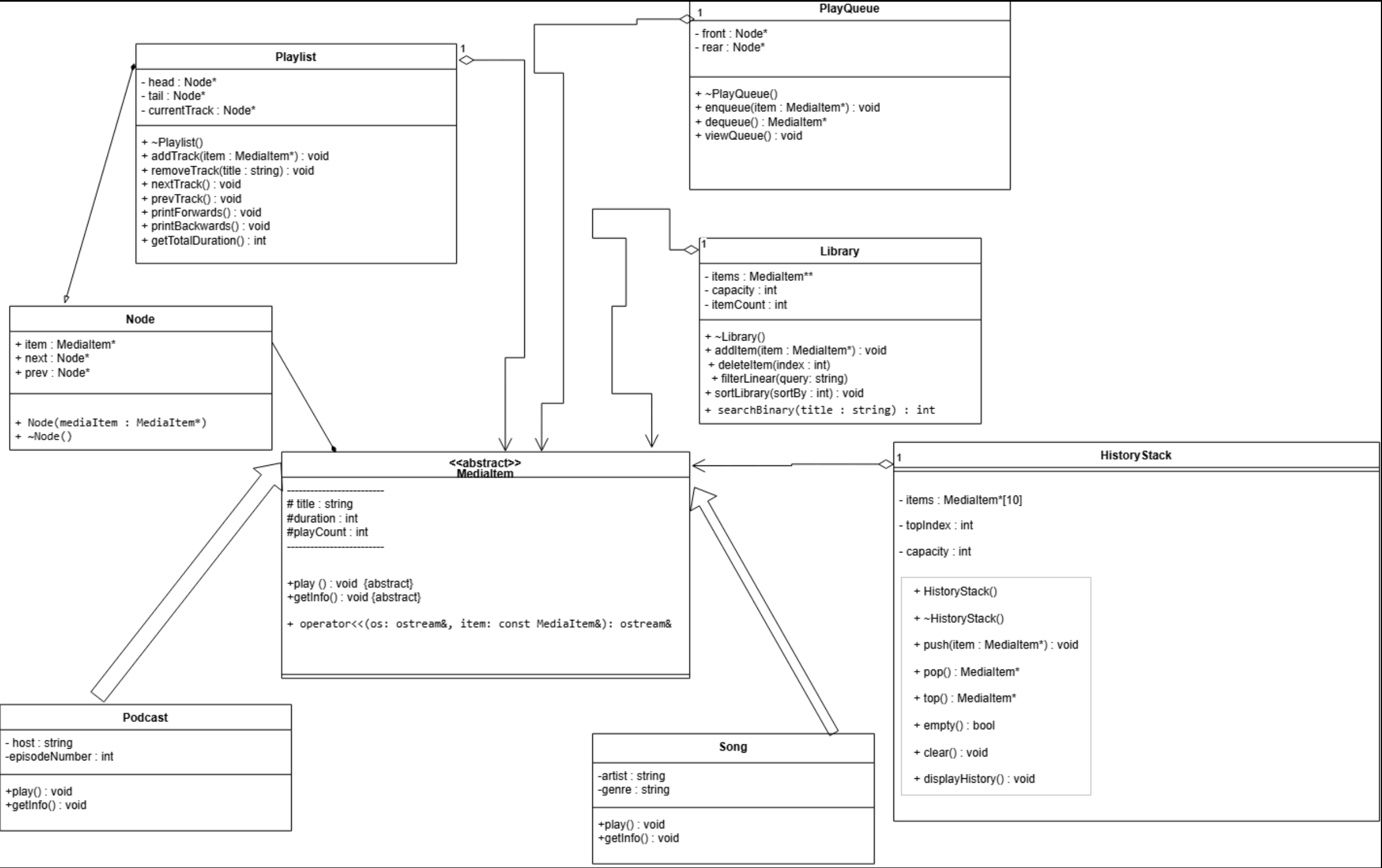
A C++ console-based music application that manages a library of songs and podcasts, custom dynamic playlists, and playback queues.

Key Features:

- Dynamic Playlist: Smooth forward and backward track navigation using a custom Doubly Linked List.
- Up Next Queue: Sequential track queueing built from scratch.
- Algorithms: Exact binary search, custom sorting with comparison counting, and recursive operations.

<https://github.com/gogonona384-crypto/PlayList-Manager>

UML-class-diagram



OOP(ARCHITECTURE)

Inheritance: Song inherits properties (title, duration) from MediaItem.

Polymorphism: Song overrides play() to stream audio logic, while Podcast overrides it for episode details.

Polymorphism Implementation: Abstract MediaItem class with virtual void play() = 0;

Inheritance: Derived classes (Song and Podcast) override play() to provide custom streaming logic.

```
include > C Song.h > ...
1  #ifndef SONG_H
2  #define SONG_H
3
4  #include "MediaItem.h"
5
6  class Song : public MediaItem {
7  private:
8      string artist;
9      string genre;
10
11  public:
12      Song(string t = "", string a = "", int d = 0, string g = "");
13
14      void play() override;
15      void getInfo() const override;
16
17      string getArtist() const;
18      string getGenre() const;
19  };
20
21 #endif
```

```
include > C MediaItem.h > MediaItem
1  #ifndef MEDIAITEM_H
2
3
4
5
6
7
8
9
10
11  class MediaItem {
12  protected:
13      string title;
14      int duration;
15      int playCount;
16
17  public:
18      MediaItem(string t = "", int d = 0) : title(t), duration(d), playCount(0) {}
19
20      virtual ~MediaItem() = default;
21
22      virtual void play() = 0;
23      virtual void getInfo() const = 0;
24
25      string getTitle() const { return title; }
26      int getDuration() const { return duration; }
27      int getPlayCount() const { return playCount; }
28      void incrementPlayCount() { playCount++; }
29
30      string getFormattedDuration() const {
31          int mins = duration / 60;
32          int secs = duration % 60;
33          ostringstream oss;
34          oss << setfill('0') << setw(2) << mins << ":"
35              << setfill('0') << setw(2) << secs;
36          return oss.str();
37      }
```

Data structure & Algorithms Complexity

Doubly Linked List (Playlist): Built from scratch to enable $O(1)$ bidirectional (next / prev) traversal.
Queue (PlayQueue): Custom FIFO structure from scratch for managing the playback queue.
Recursion: Used for calculating totalDuration and printBackwards

Operation	Structure	Time Complexity
Search by Title	Sorted Array	$O(\log n)$ (Binary Search)
Sort Library	Array	$O(n^2)$ (Selection Sort)
Enqueue / Dequeue	Custom Queue	$O(1)$
Traverse Tracks	Doubly Linked List	$O(1)$ per step

Live Demo & Conclusion

Interactive Walkthrough:

Library Management: Add & view songs/podcasts.

Playlist Traversal: Demonstrate next and previous navigation on Doubly Linked List.

Up Next Queue: Add and play tracks sequentially from custom Queue.

Search & Sort: Execute Binary Search and Selection Sort with comparison metrics.

Stats & Recursion: Show total library duration computed recursively.

ThankYou